



Deploying a Secure Netflix Clone with DevSecOps and Kubernetes



Project Overview

This project demonstrates a complete DevSecOps implementation for deploying a Netflix Clone application using modern cloud-native technologies. The deployment integrates security scanning, CI/CD automation, monitoring, Kubernetes orchestration, and GitOps practices to create a production-ready software delivery pipeline.



Technology Stack

Category	Tools
Cloud Platform	AWS EC2, Amazon EKS
Containerization	Docker
Source Control	GitHub
CI/CD	Jenkins
Code Quality	SonarQube
Vulnerability Scanning	Trivy, OWASP Dependency Check
Monitoring	Prometheus, Grafana, Node Exporter
GitOps	Argo CD
Application	Netflix
External API	TMDB API



Table of Contents:-



Phase 1: AWS Infrastructure Provisioning



Phase 2: Security Group & Application Configuration



Phase 3: Docker Build & Application Deployment



Phase 4: DevSecOps Security Integration



Phase 5: Jenkins CI/CD Pipeline Automation



Phase 6: Monitoring & Observability Setup



Phase 7: Notification & Alerting Configuration



Phase 8: Amazon EKS Cluster Deployment



Phase 9: GitOps Implementation using Argo CD



Phase 10: Final Deployment Verification & Production Validation

Phase 1: AWS Infrastructure Provisioning

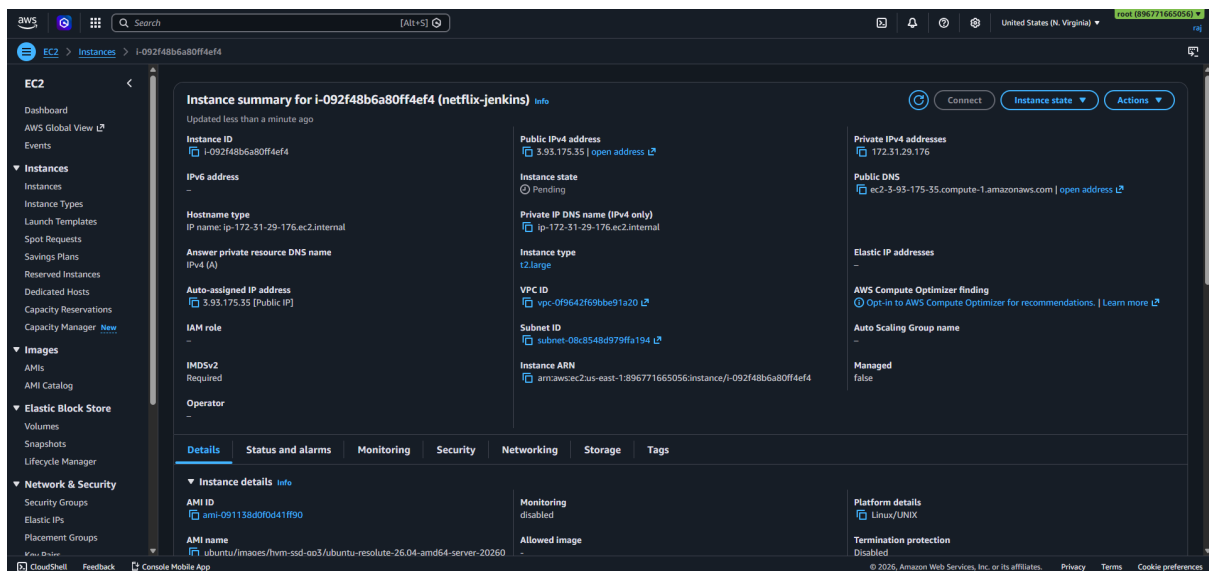
Step 1: Launch EC2 Instances

Purpose

Provision AWS infrastructure required for hosting Jenkins, SonarQube, Prometheus, Grafana, Argo CD, and the Netflix application.

Implementation Steps

1. Login to AWS Management Console.
2. Navigate to EC2 Dashboard.
3. Launch Ubuntu-based EC2 instances.
4. Select the desired instance type.
5. Configure networking.
6. Create or select a Key Pair.
7. Launch the instances.



Outcome

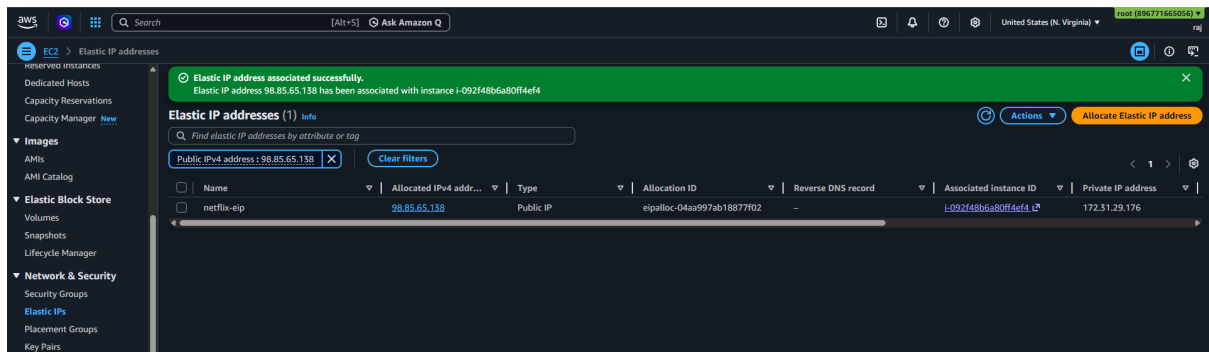
 EC2 instances successfully created and available for configuration.

Step 2: Associate Elastic IP Addresses

Purpose: Provide static public IP addresses for consistent external access.

Implementation Steps

1. Open EC2 Dashboard.
2. Navigate to Elastic IPs.
3. Allocate a new Elastic IP.
4. Associate the Elastic IP with the target instance.



Outcome:  Public static IPs assigned successfully.

Step 3: Connect Using Git Bash

Purpose: Establish SSH connectivity to the EC2 server.

Implementation Steps

```
ubuntu@ip-172-31-29-176:~$ ssh netflix-lab
Welcome to Ubuntu 26.04 LTS (GNU/Linux 7.0.0-1004-aws x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Thu Jun 11 07:51:17 UTC 2026

System load: 0.07          Processes:      126
Usage of /:  8.8% of 23.10GB Users logged in:  0
Memory usage: 3%          IPv4 address for enx0: 172.31.29.176
Swap usage:  0%

Expanded Security Maintenance for Applications is not enabled.
0 updates can be applied immediately.
Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

Last login: Thu Jun 11 07:50:07 2026 from 119.82.104.90
ubuntu@ip-172-31-29-176:~$ uname -a
Linux ip-172-31-29-176 7.0.0-1004-aws #4-Ubuntu SMP PREEMPT Mon Apr 13 13:14:24 UTC 2026 x86_64 GNU/Linux
ubuntu@ip-172-31-29-176:~$
```

Outcome:  Secure shell access established.

Phase 2: Network & Application Configuration

Step 4: Update Security Groups

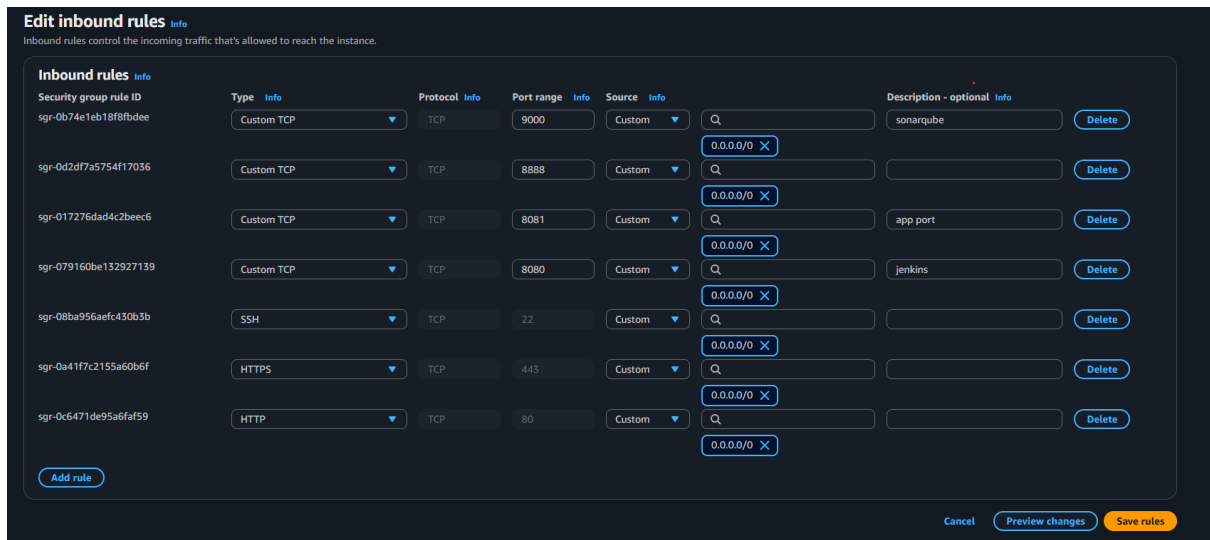
Purpose

Allow traffic required by Jenkins, SonarQube, Prometheus, Grafana, Node Exporter, Argo CD, and the application.

Implementation Steps

Configure inbound rules:

- SSH (22)
- HTTP (80)
- HTTPS (443)
- Jenkins (8080)
- SonarQube (9000)
- Prometheus (9090)
- Grafana (3000)
- Node Exporter (9100)



Security group rule ID	Type	Protocol	Port range	Source	Description - optional	Info
sgr-0b74e1eb18f8fdee	Custom TCP	TCP	9000	Custom	sonarqube	0.0.0.0/0 X
sgr-0d2df7a5754f17036	Custom TCP	TCP	8888	Custom		0.0.0.0/0 X
sgr-017276dad4c2beec6	Custom TCP	TCP	8081	Custom	app port	0.0.0.0/0 X
sgr-079160be132927139	Custom TCP	TCP	8080	Custom	jenkins	0.0.0.0/0 X
sgr-08ba95aefc430b3b	SSH	TCP	22	Custom		0.0.0.0/0 X
sgr-0a41f7c2155a60b6f	HTTPS	TCP	443	Custom		0.0.0.0/0 X
sgr-0c6471de95a6faf59	HTTP	TCP	80	Custom		0.0.0.0/0 X

Outcome

 Required services accessible externally.

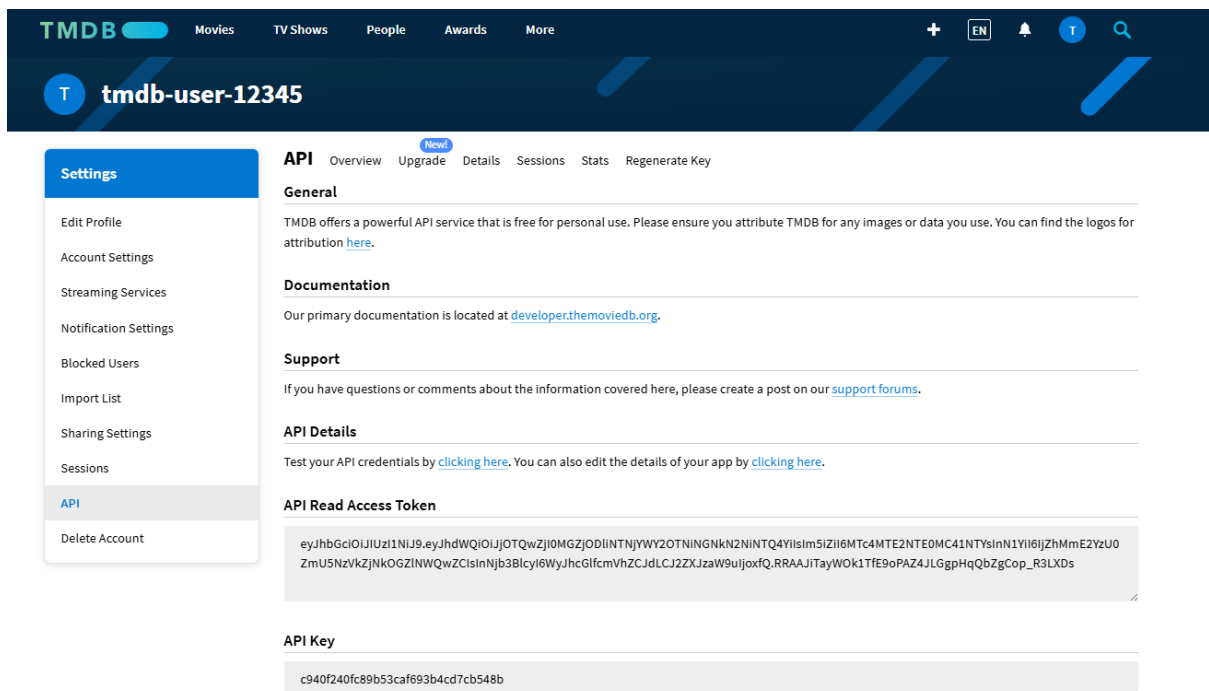
Step 5: Configure TMDB API

Purpose

Enable movie metadata integration for the Netflix Clone application.

Implementation Steps

1. Create TMDB account.
2. Generate API Key.
3. Copy API credentials.
4. Use API key during Docker build.



The screenshot shows the TMDB API configuration page. The 'API' section is active, showing the 'General' tab. The 'General' section provides information about the TMDB API service. The 'Documentation' section points to the developer.themoviedb.org website. The 'Support' section provides a link to the support forums. The 'API Details' section provides a link to test the API credentials. The 'API Read Access Token' section displays a long alphanumeric string. The 'API Key' section displays a shorter alphanumeric string.

Docker Build Command

```
ubuntu@ip-172-31-29-176:~/DevSecOps-Project$ docker run -d --name netflix -p 8888:80 netflix
b05646f828f889e168f12c0b3808cf09a12e9a6e5d14b5b9259a0141efc8d5dd
ubuntu@ip-172-31-29-176:~/DevSecOps-Project$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED      STATUS      PORTS
b05646f828f8   netflix   "nginx -g 'daemon of..." 8 seconds ago Up 7 seconds 0.0.0.0:8888->80/tcp, [::]:8888->80/tcp netflix
ubuntu@ip-172-31-29-176:~/DevSecOps-Project$
```

Outcome

 Application successfully integrated with TMDB API.

Phase 3: Docker Deployment

Step 6: Build and Run Netflix Clone

Purpose

Containerize and deploy the application locally.

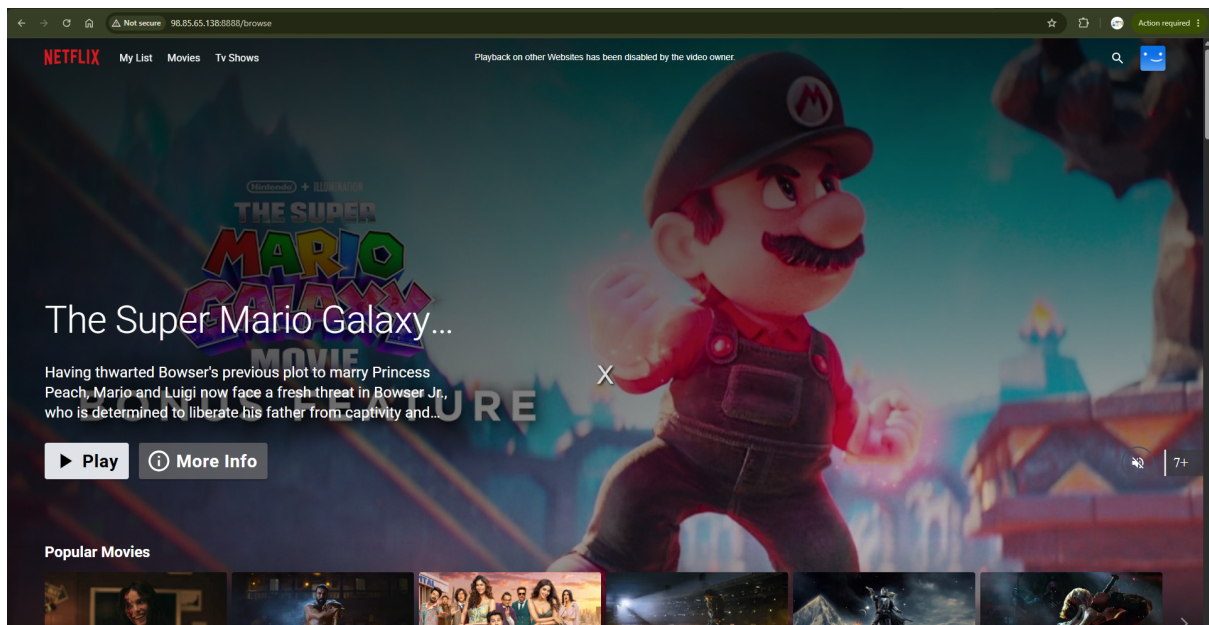
Implementation Steps

Build Docker image:

```
docker build -t netflix .
```

Run Docker container:

```
docker run -d -p 8888:80 netflix
```



Outcome

 Netflix Clone successfully running on port 8888.



Phase 4: DevSecOps Security Integration



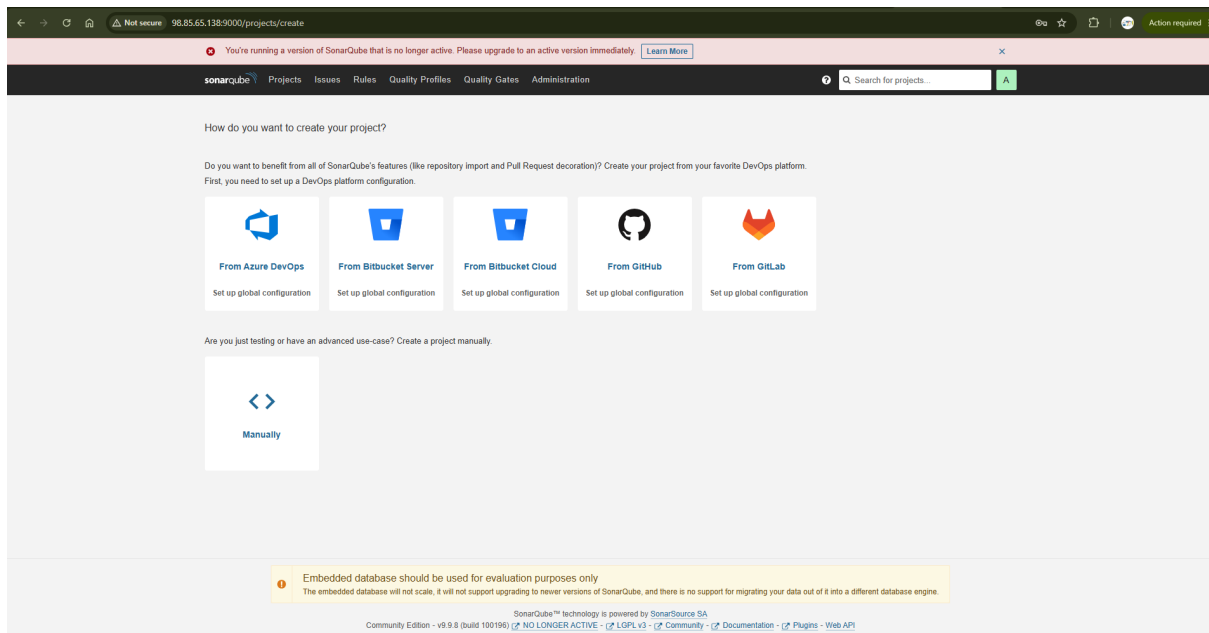
Step 7: Install SonarQube

Purpose

Perform static code analysis and quality validation.

Implementation Steps

```
docker run -d \  
--name sonar \  
-p 9000:9000 \  
sonarqube:lts-community
```



Outcome



SonarQube dashboard available on port 9000.

Purpose

Identify vulnerabilities in source code and container images.

Implementation Steps

Filesystem scan:

```
trivy fs .
```

```
ubuntu@ip-172-31-29-176: ~/DevSecOps-Project
ubuntu@ip-172-31-29-176:~/DevSecOps-Project$ trivy --version
Version: 0.52.2
ubuntu@ip-172-31-29-176:~/DevSecOps-Project$ AC
ubuntu@ip-172-31-29-176:~/DevSecOps-Project$ ls
Dockerfile README.md package.json public tsconfig.json vercel.json yarn.lock
Kubernetes index.html pipeline.txt src tsconfig.node.json vite.config.ts
ubuntu@ip-172-31-29-176:~/DevSecOps-Project$ trivy fs .
2026-06-11T08:57:40Z INFO Need to update DB
2026-06-11T08:57:40Z INFO Downloading DB... repository="ghcr.io/aquasecurity/trivy-db:2"
95.94 MiB / 95.94 MiB [-----] 100.00% 22.49 MiB p/s 4.5s
2026-06-11T08:57:45Z INFO Vulnerability scanning is enabled
2026-06-11T08:57:45Z INFO Secret scanning is enabled
2026-06-11T08:57:45Z INFO If your scanning is slow, please try '--scanners vuln' to disable secret scanning
2026-06-11T08:57:45Z INFO Please see also https://aquasecurity.github.io/trivy/v0.52/docs/scanner/secret/#recommendation for faster secret detection
2026-06-11T08:57:45Z INFO Suppressing dependencies for development and testing. To display them, try the '--include-dev-deps' flag.
2026-06-11T08:57:45Z INFO Number of language-specific files num=1
2026-06-11T08:57:45Z INFO [yarn] Detecting vulnerabilities...
yarn.lock (yarn)
Total: 18 (UNKNOWN: 0, LOW: 2, MEDIUM: 6, HIGH: 10, CRITICAL: 0)
```

Library	Vulnerability	Severity	Status	Installed Version
Fixed Version	Title			
@babel/runtime-alpha.17	CVE-2025-27789	MEDIUM	fixed	7.20.0
ted code	Babel has inefficient RegExp complexity in general with .replace when transpiling...			7.26.10, 8.0.0
				https://avd.aquasec.com/nvd/cve-2025-27789

Container scan:

```
trivy image netflix
```

```
ubuntu@ip-172-31-29-176:~/DevSecOps-Project$ docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
netflix:latest	7630672f45ea	110MB	34.3MB	U
nginx:stable-alpine	5f979dcfed4c	93.6MB	26.9MB	
node:16.17.0-alpine	2c405ed42fc0	168MB	41.6MB	
sonarqube:lts-community	f709975ab31d	1.02GB	398MB	U

```
ubuntu@ip-172-31-29-176:~/DevSecOps-Project$ trivy image netflix:latest
2026-06-11T09:08:31Z INFO [vuln] Need to update DB
2026-06-11T09:08:31Z INFO [vuln] Downloading vulnerability DB...
2026-06-11T09:08:31Z INFO [vuln] Downloading artifact... repo="mirror.gcr.io/aquasec/trivy-db:2"
95.94 MiB / 95.94 MiB [-----] 100.00% 25.33 MiB p/s 4.0s
2026-06-11T09:08:36Z INFO [vuln] Artifact successfully downloaded repo="mirror.gcr.io/aquasec/trivy-db:2"
2026-06-11T09:08:36Z INFO [vuln] vulnerability scanning is enabled
2026-06-11T09:08:36Z INFO [secret] Secret scanning is enabled
2026-06-11T09:08:36Z INFO [secret] If your scanning is slow, please try '--scanners vuln' to disable secret scanning
2026-06-11T09:08:36Z INFO [secret] Please see https://trivy.dev/docs/v0.71/guide/scanner/secret#recommendation for faster secret detection
2026-06-11T09:08:38Z INFO Detected OS family="alpine" version="3.23.4"
2026-06-11T09:08:38Z INFO [alpine] Detecting vulnerabilities... os_version="3.23" repository="3.23" pkg_num=71
2026-06-11T09:08:39Z INFO Number of language-specific files num=0
2026-06-11T09:08:39Z WARN Using severities from other vendors for some vulnerabilities.
Read https://trivy.dev/docs/v0.71/guide/scanner/vulnerability#severity-selection for details
.
```

Report Summary

Target	Type	Vulnerabilities	Secrets
netflix:latest (alpine 3.23.4)	alpine	31	-

Outcome

✓ Security vulnerabilities detected and reported.



Phase 5: Jenkins CI/CD Automation



Step 9: Install Jenkins

Purpose

Automate build, test, scan, and deployment workflows.

Implementation Steps

Install Jenkins and verify dashboard access.

The screenshot shows the Jenkins web dashboard on the left and a terminal window on the right. The dashboard displays the 'Welcome to Jenkins!' message and options to 'Create a job', 'Set up a distributed build', 'Configure a cloud', and 'Learn more about distributed builds'. The terminal window shows the installation of Jenkins on an Ubuntu system. It includes commands like 'sudo systemctl status jenkins' and 'sudo cat /var/lib/jenkins/secrets/initialAdminPassword', along with their outputs showing the service is active and running.

Outcome

✅ Jenkins server operational.



Step 10: Install Jenkins Plugins

Required Plugins

- Eclipse Temurin Installer
- SonarQube Scanner
- NodeJS Plugin
- Email Extension Plugin

Outcome

✅ Jenkins extended with required capabilities.

Step 11: Configure Jenkins Tools

Implementation Steps

Configure:

- JDK 17
- NodeJS 16
- SonarQube Scanner

Create Sonar Token and store in Jenkins Credentials.

 **Jenkins** / Manage Jenkins / Credentials / System / Global credentials (unrestricted)

New credentials

Kind

Secret text

Scope ?

Global (Jenkins, nodes, items, all child items, etc)

Secret

.....

ID ?

Sonar-Token

Description ?

Sonar-Token

Create

Added sonar server in Jenkins

 **Jenkins** / Manage Jenkins / System

SonarQube servers

If checked, job administrators will be able to inject a SonarQube server configuration as environment variables in the build.

☐ Environment variables

SonarQube installations

List of SonarQube installations

Name

sonar-server

Server URL

Default is http://localhost:9000

http://90.85.65.138:9000/

Server authentication token

SonarQube authentication token. Mandatory when anonymous access is disabled.

Sonar-token

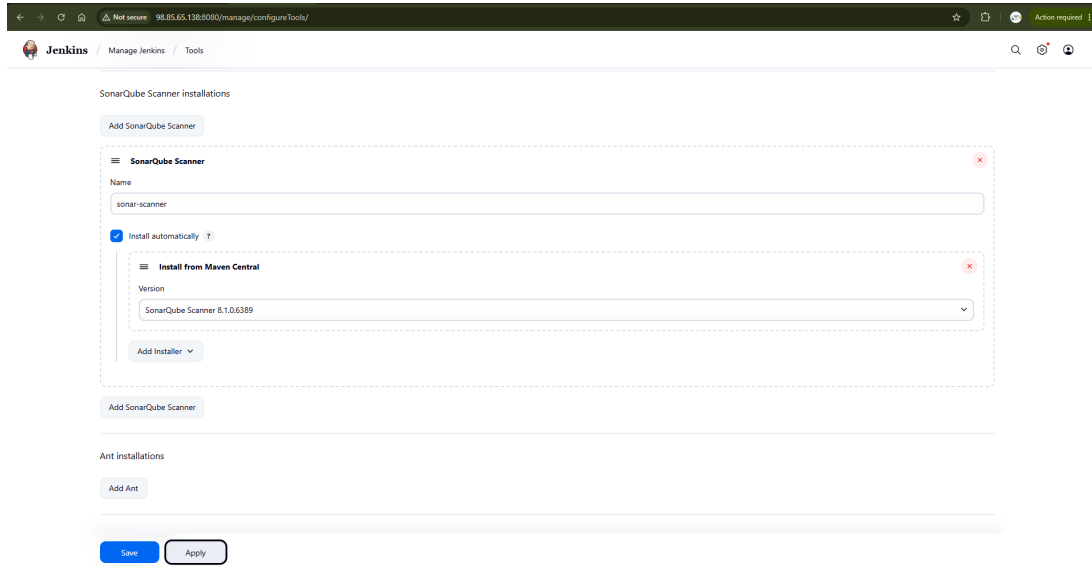
+ Add

Advanced

Add SonarQube

Save Apply

SonarQube Scanner installations



Outcome

✓ Jenkins successfully integrated with SonarQube.

Step 12: Execute First Pipeline Build

Purpose

Validate CI/CD workflow execution.

Stage	Declarative: Tool Install	clean workspace	Checkout from Git	Sonarqube Analysis	quality gate	Install Dependencies
Average stage times: (Full run time: ~47s)	233ms	407ms	1s	25s	519ms	19s
Jun 11 15:14	233ms	407ms	1s	25s	519ms	19s

SonarQube Quality Gate

Netflix **Passed**

server-side processing: **Success**

Permalinks

- Last build (#2), 11 min ago
- Last stable build (#2), 11 min ago
- Last successful build (#2), 11 min ago
- Last completed build (#2), 11 min ago

Outcome

✓ Initial pipeline executed successfully.

Step 13: Install Dependency Check & Docker Tools

Purpose: Add software composition analysis and container automation.

 **Jenkins** / Manage Jenkins / Tools

Dependency-Check installations

Add Dependency-Check

≡ Dependency-Check

Name

DP-Check

☒ Install automatically ?

≡ Install from github.com

Version

dependency-check 12.2.2

Add Installer ▾

Add Dependency-Check

Docker installations

Add Docker

≡ Docker

Name

docker

☒ Install automatically ?

≡ Download from docker.com

Docker version ?

latest

Add Installer ▾

Save

Apply

Outcome

 Dependency scanning integrated.



Step 14: Update Jenkins Pipeline

Purpose: Integrate quality gates, security scanning, and Docker publishing.

```

pipeline{
  agent any
  tools{
    jdk 'jdk17'
    nodejs 'node18'
  }
  environment {
    SCANNER_HOME=tool 'sonar-scanner'
  }
  stages {
    stage('clean workspace'){
      steps{
        cleanWs()
      }
    }

    stage('Checkout from Git'){
      steps{
        git branch: 'main', url: 'https://github.com/W4si/DevSecOps-Project.git'
      }
    }

    stage('Sonarqube Analysis '){
      steps{
        withSonarQubeEnv('sonar-server') {
          sh ''' $SCANNER_HOME/bin/sonar-scanner -Dsonar.projectName=Netflix \
            -Dsonar.projectKey=Netflix '''
        }
      }
    }

    stage('Quality gate'){
      steps {
        script {
          waitForQualityGate shortPipeline: false, credentialsId: 'Sonar-token'
        }
      }
    }

    stage('Install Dependencies') {
      steps {
        sh "npm install"
      }
    }

    stage('OWASP FS SCAN') {
      steps {
        dependencyCheck additionalArguments: '--scan ./ --disableWarnAudit --disableNodeAudit', odcInstallation: 'DP-Check'
        dependencyCheckPublisher pattern: '**/dependency-check-report.xml'
      }
    }

    stage('TRIVY FS SCAN') {
      steps {
        sh "trivy fs . > trivyfs.txt"
      }
    }

    stage('Patch Dockerfile') {
      steps {
        sh '''
        sed -i 's/node:18.17.0-alpine/node:20-alpine/g' Dockerfile
        grep "FROM node" Dockerfile
        '''
      }
    }

    stage('Docker Build & Push'){
      steps{
        script{
          withDockerRegistry(credentialsId: 'docker'){
            sh "docker build --build-arg TRIM_V3_API_KEY=c9402240c0c059b53ca2092b4cd7cb540b -t netflix ."
            sh "docker tag netflix raj562k/netflix:latest"
            sh "docker push raj562k/netflix:latest"
          }
        }
      }
    }

    stage('TRIVY'){
      steps{
        sh "trivy image raj562k/netflix:latest > trivyimage.txt"
      }
    }

    stage('Deploy to container'){
      steps{
        sh '''
        docker rm -f netflix || true
        docker run -d --name netflix -p 8888:80 raj562k/netflix:latest
        '''
      }
    }
  }
}

```

Outcome

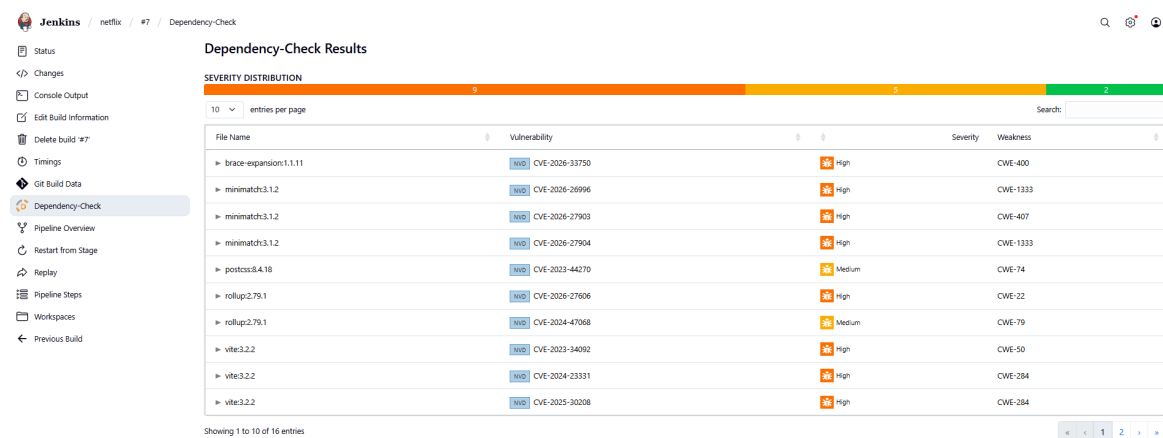
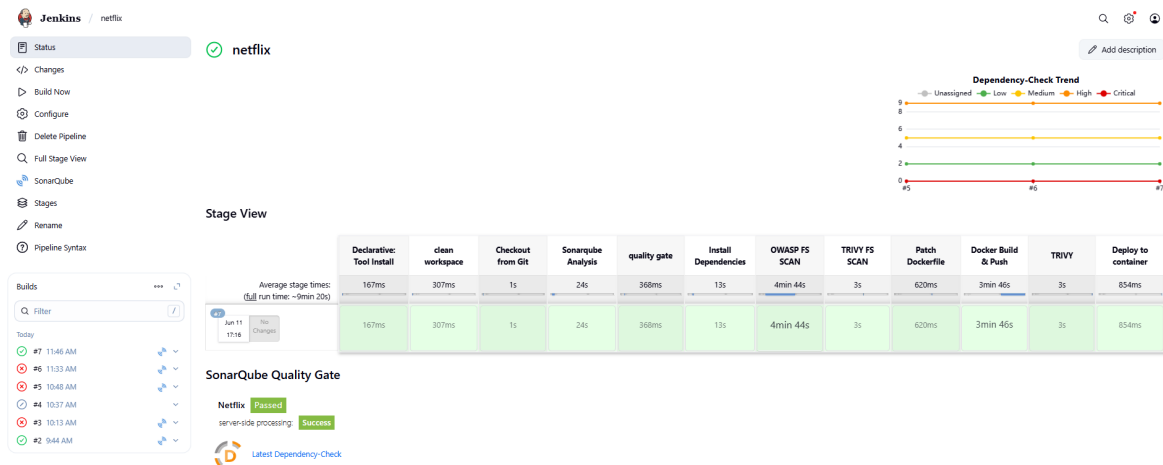


Complete DevSecOps workflow automated.

Step 15: Successful Build & Docker Hub Push

Validation

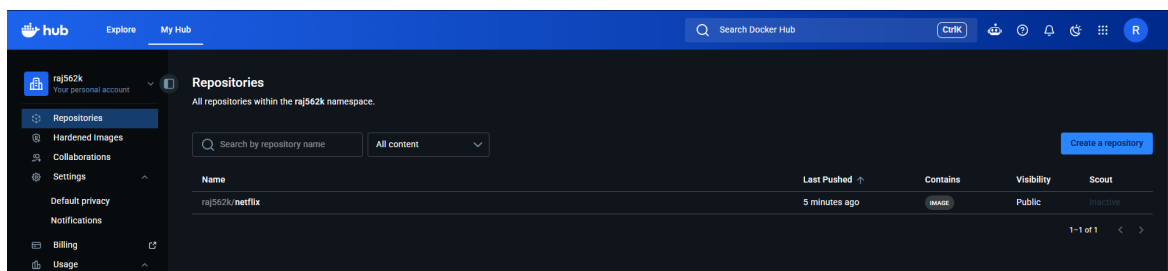
- Build Success
- Dependency Check Success
- Docker Push Success



The Jenkins interface shows the 'netflix' pipeline status. The pipeline is green, indicating success. The 'Dependency-Check Results' table shows the following results:

File Name	Vulnerability	Severity	Weakness
» brace-expansion:1.1.11	CVE-2026-33750	High	CWE-400
» minimatch3.1.2	CVE-2026-26996	High	CWE-1333
» minimatch3.1.2	CVE-2026-27903	High	CWE-407
» minimatch3.1.2	CVE-2026-27904	High	CWE-1333
» postcss:8.4.18	CVE-2023-44270	Medium	CWE-74
» rollup:2.79.1	CVE-2026-27606	High	CWE-22
» rollup:2.79.1	CVE-2024-47068	Medium	CWE-79
» vite3.2.2	CVE-2023-34092	High	CWE-50
» vite3.2.2	CVE-2024-23331	High	CWE-284
» vite3.2.2	CVE-2025-30208	High	CWE-284

Showing 1 to 10 of 16 entries



Outcome

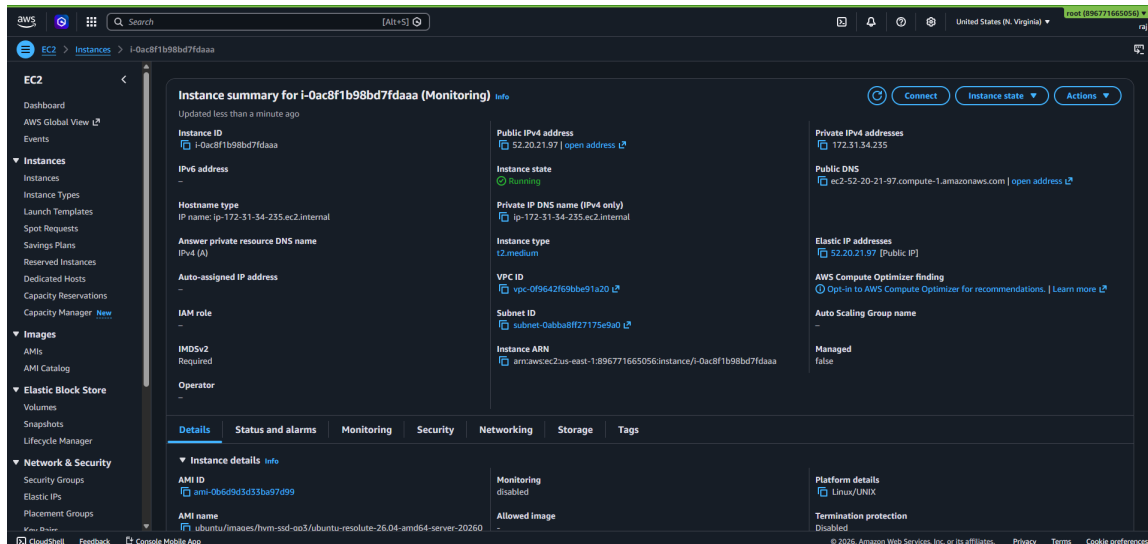
 Application image published successfully.

Phase 6: Monitoring & Observability

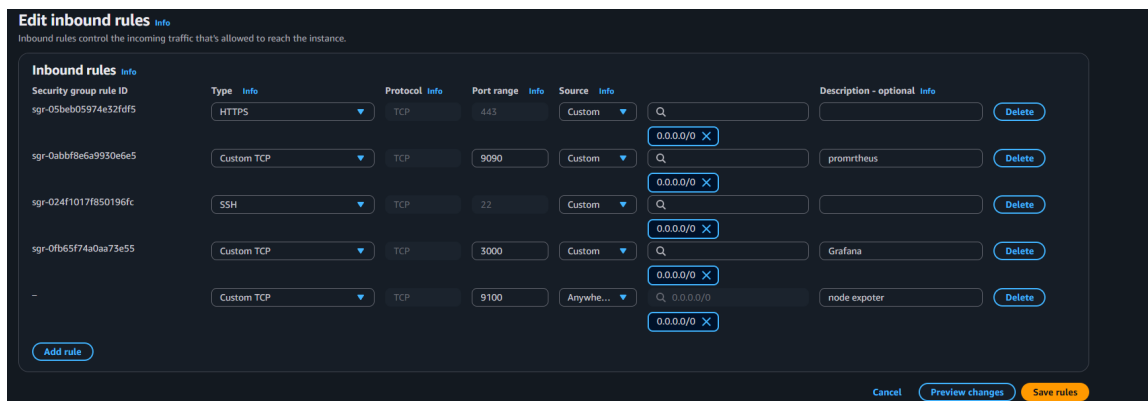
Step 16: Launch EC2 Instances and Install Prometheus

Purpose: Collect infrastructure and application metrics.

Created EC2 and Associated the elastic IP

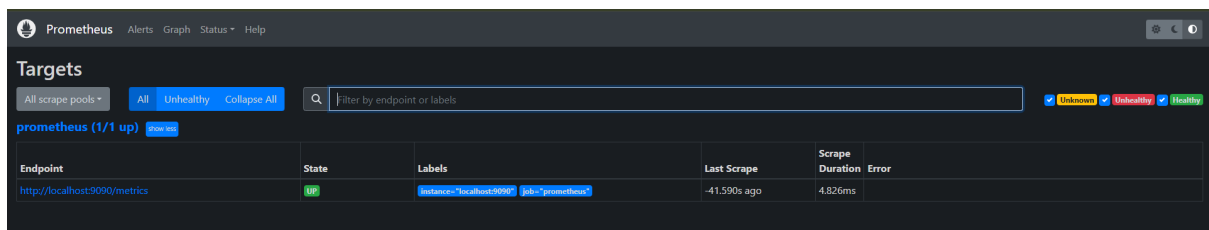


Security Group



Outcome

✓ I can access Prometheus in a web browser using your server's IP and port 9090: <http://52.20.21.97:9090/>



Step 17: Install Node Exporter

Purpose

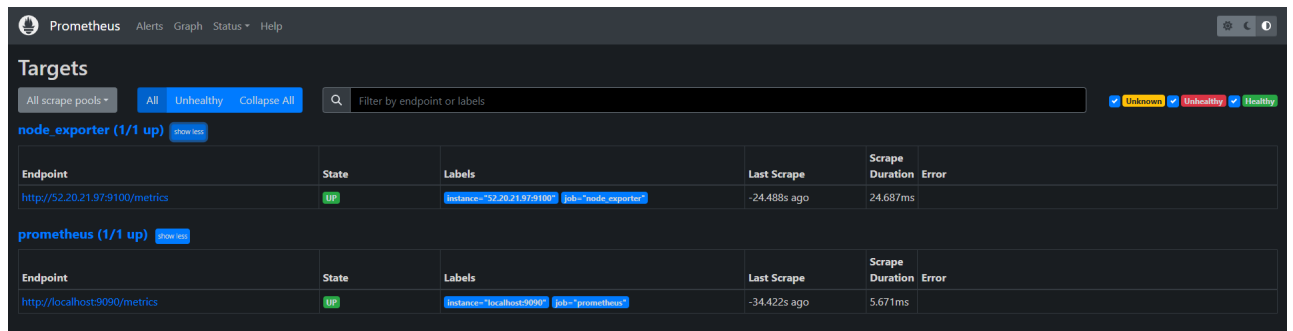
Expose server metrics to Prometheus.

```
ubuntu@ip-172-31-34-235:~$ sudo systemctl enable node_exporter
sudo systemctl start node_exporter
Created symlink '/etc/systemd/system/multi-user.target.wants/node_exporter.service' -> '/etc/systemd/system/node_exporter.service'.
ubuntu@ip-172-31-34-235:~$ sudo systemctl status node_exporter
● node_exporter.service - Node Exporter
   Loaded: loaded (/etc/systemd/system/node_exporter.service; enabled; preset: enabled)
   Active: active (running) since Sat 2026-06-13 07:41:59 UTC; 3s ago
     Invocation: 49b79cba0ddf45809545fe01874cf483
    Main PID: 3947 (node_exporter)
      Tasks: 4 (limit: 3807)
     Memory: 2.3M (peak: 2.6M)
        CPU: 14ms
    CGroup: /system.slice/node_exporter.service
            └─3947 /usr/local/bin/node_exporter --collector.logind
```

Node Exporter up and running.

Note: make sure to enable 9100 port in SG

In prometheus target section “node exporter” has been added



The screenshot shows the Prometheus Targets page. It lists two targets: 'node exporter (1/1 up)' and 'prometheus (1/1 up)'. The 'node exporter' target is at 'http://52.20.21.97:9100/metrics' and is in an 'UP' state. The 'prometheus' target is at 'http://localhost:9090/metrics' and is also in an 'UP' state. Both targets have labels indicating their instance and job names.

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
node exporter (1/1 up)					
http://52.20.21.97:9100/metrics	UP	instance="52.20.21.97:9100" job="node_exporter"	-24.488s ago	24.687ms	
prometheus (1/1 up)					
http://localhost:9090/metrics	UP	instance="localhost:9090" job="prometheus"	-34.422s ago	5.671ms	

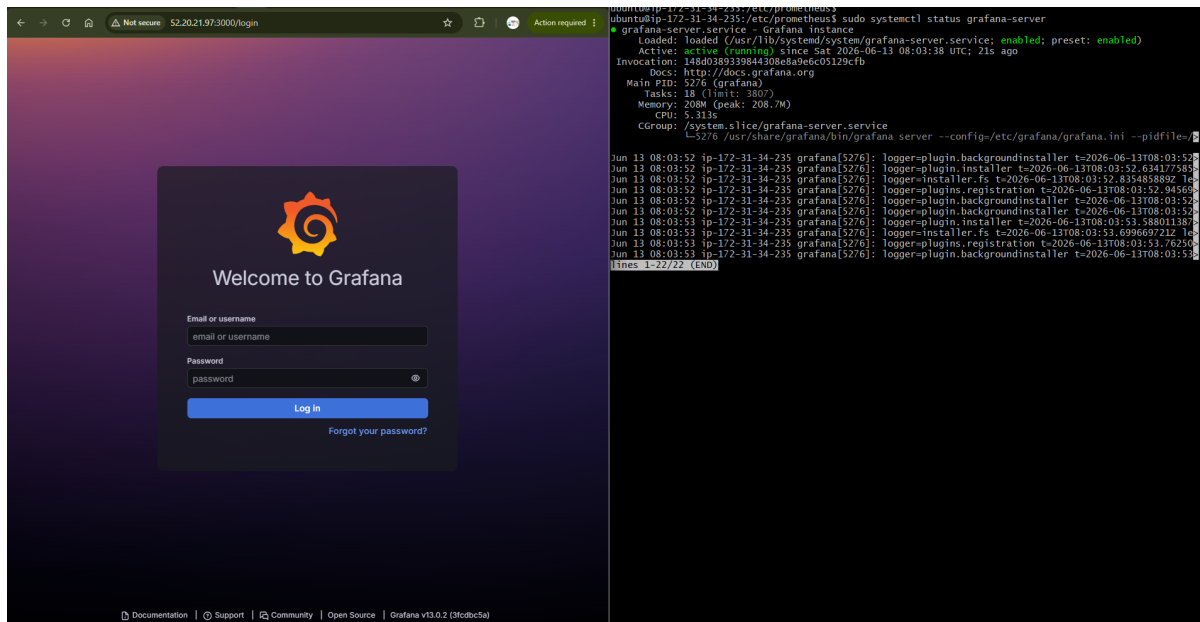
Outcome

✅ Host metrics available for monitoring.

Step 18: Install Grafana

Purpose

Visualize monitoring metrics.



The screenshot shows the Grafana login interface on the left and a terminal window on the right. The login page has a 'Welcome to Grafana' message, input fields for 'Email or username' and 'Password', a 'Log in' button, and a 'Forgot your password?' link. The terminal window displays the command to start the Grafana service and its status output, showing it is active and running.

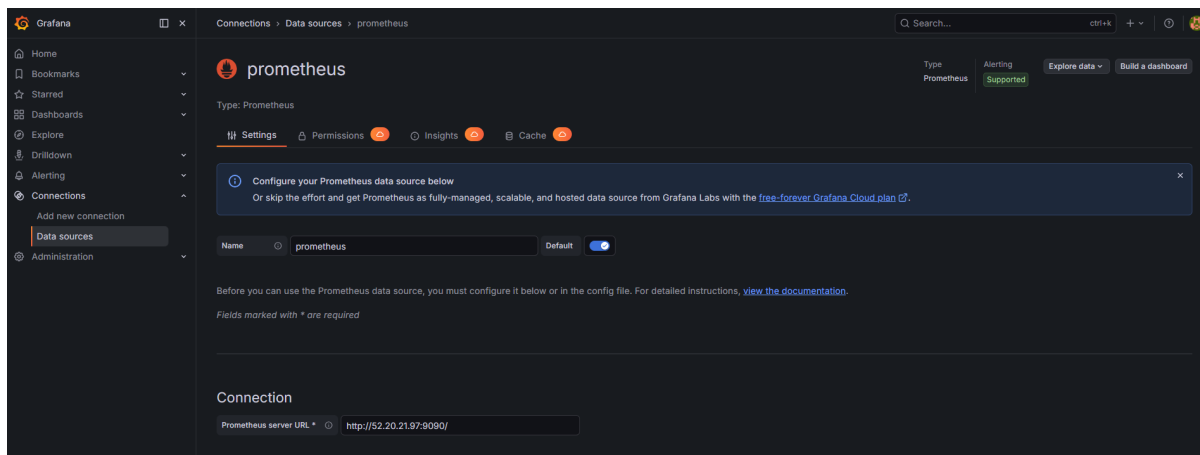
Outcome

✅ Grafana dashboard available.

Step 19: Configure Dashboards

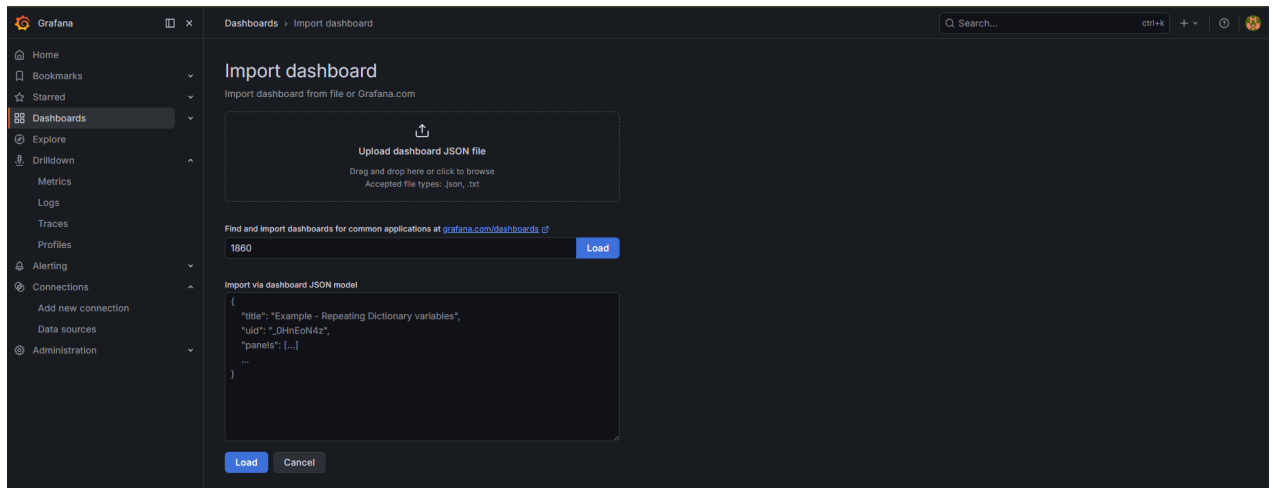
Implementation

- Add Prometheus datasource



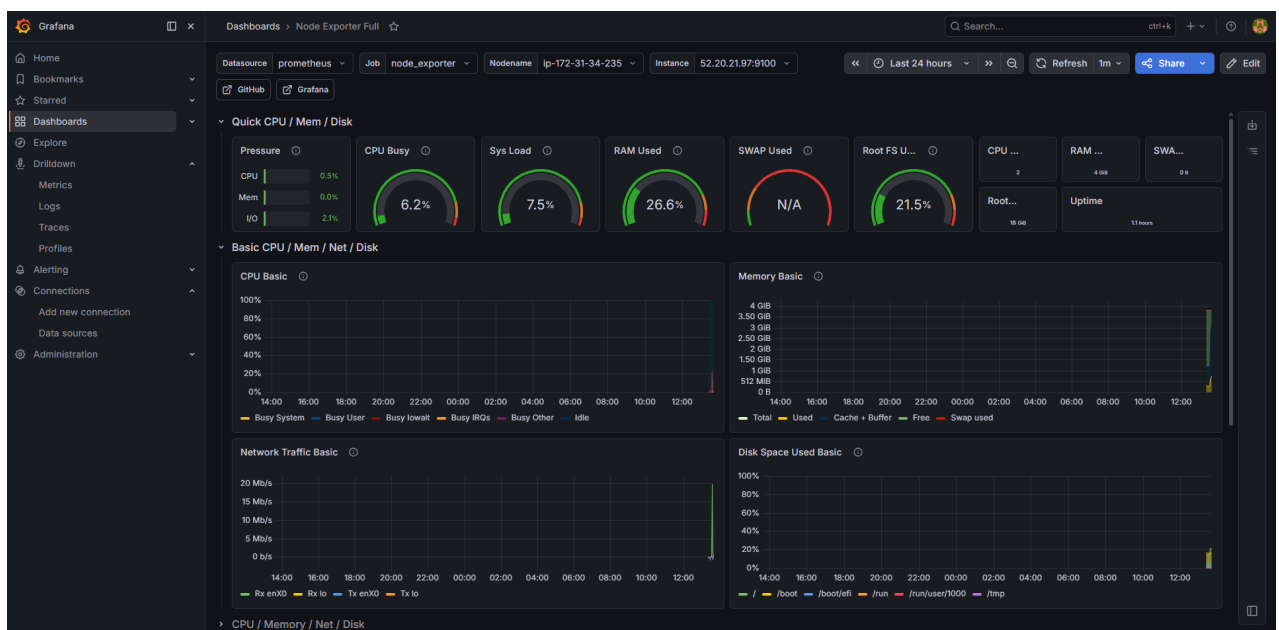
The screenshot shows the Grafana web interface with the 'Connections > Data sources > prometheus' path selected. The 'prometheus' data source is configured with the name 'prometheus' and the URL 'http://52.20.21.97:9090/'. A message prompts the user to configure the data source or skip the effort by using Grafana Cloud. The 'Connection' section shows the 'Prometheus server URL' field filled with the provided URL.

- Import Node Exporter Dashboard (ID: 1860)



Outcome

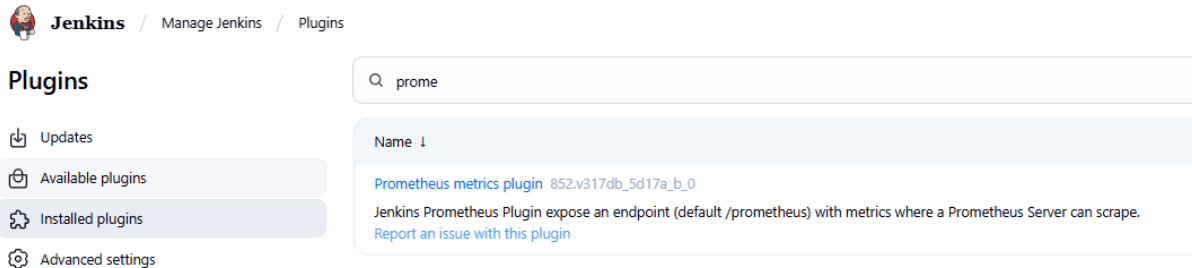
✓ Infrastructure dashboards available.



Step 20: Monitor Jenkins

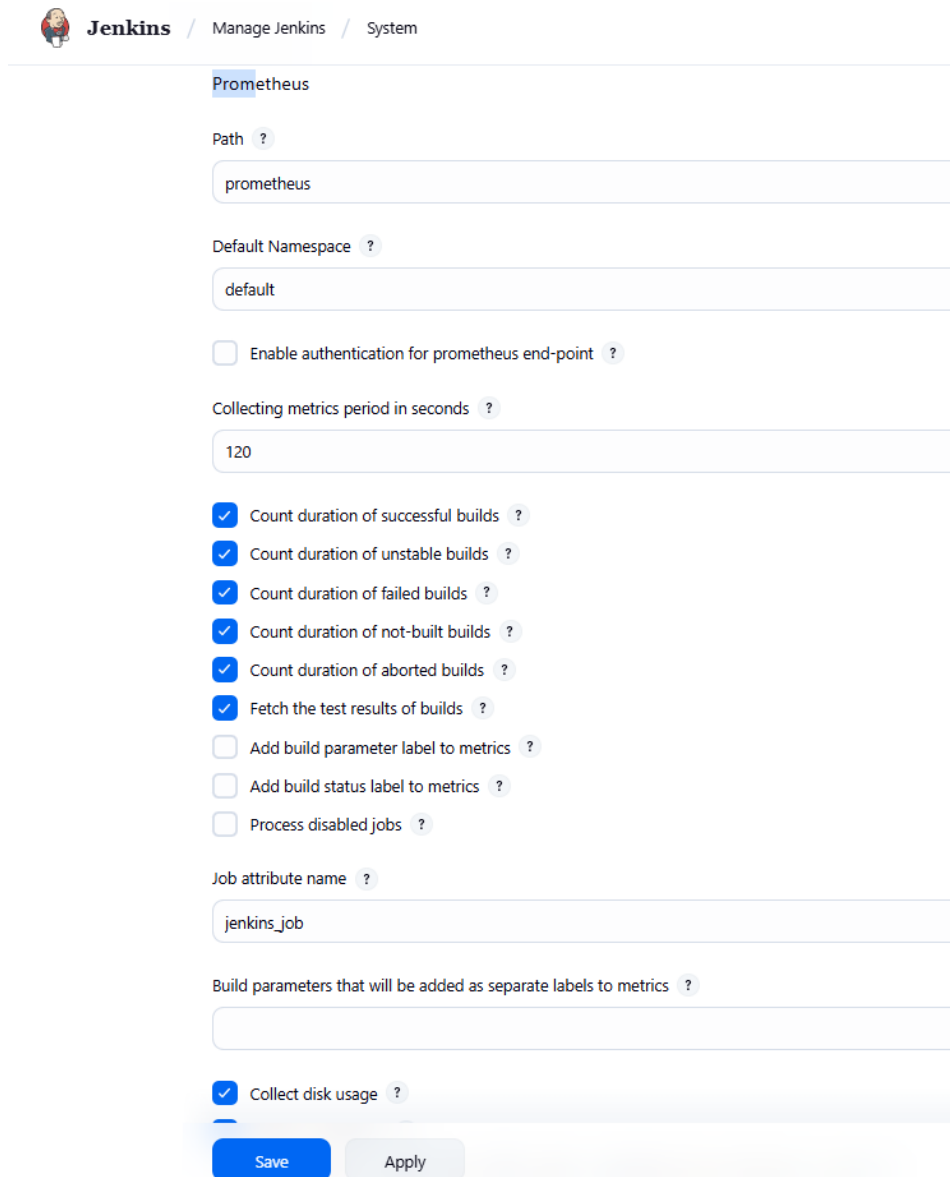
Implementation

- Install Prometheus Plugin



The screenshot shows the Jenkins 'Manage Jenkins' page, specifically the 'Plugins' section. The breadcrumb trail is 'Jenkins / Manage Jenkins / Plugins'. On the left, there are links for 'Updates', 'Available plugins', 'Installed plugins' (which is highlighted), and 'Advanced settings'. A search bar contains the text 'prome'. Below the search bar, a list of plugins is shown, with 'Prometheus metrics plugin' selected. The plugin details indicate version '852.v317db_5d17a_b_0' and provide a description: 'Jenkins Prometheus Plugin expose an endpoint (default /prometheus) with metrics where a Prometheus Server can scrape.' There is also a link to 'Report an issue with this plugin'.

- Setting up Prometheus server



The screenshot shows the Jenkins 'System' configuration page for the 'Prometheus' plugin. The breadcrumb trail is 'Jenkins / Manage Jenkins / System'. The 'Prometheus' section is active. The 'Path' field is set to 'prometheus'. The 'Default Namespace' is set to 'default'. The checkbox 'Enable authentication for prometheus end-point' is unchecked. The 'Collecting metrics period in seconds' is set to '120'. There are several checkboxes for metrics collection, all of which are checked: 'Count duration of successful builds', 'Count duration of unstable builds', 'Count duration of failed builds', 'Count duration of not-built builds', 'Count duration of aborted builds', and 'Fetch the test results of builds'. There are also three unchecked checkboxes: 'Add build parameter label to metrics', 'Add build status label to metrics', and 'Process disabled jobs'. The 'Job attribute name' is set to 'jenkins_job'. The 'Build parameters that will be added as separate labels to metrics' field is empty. The 'Collect disk usage' checkbox is checked. At the bottom, there are 'Save' and 'Apply' buttons.

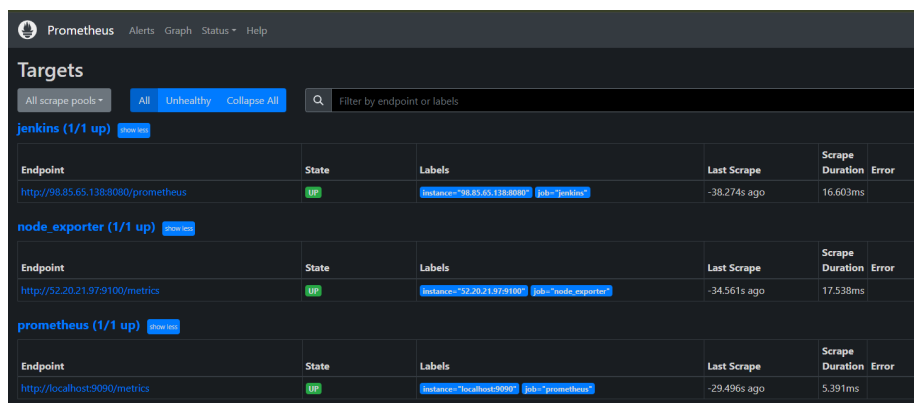
- Add Jenkins job in Prometheus - /etc/Prometheus/Prometheus.yml file

```
scrape_configs:
  - job_name: "prometheus"
    static_configs:
      - targets: ["localhost:9090"]

  - job_name: "node_exporter"
    static_configs:
      - targets: ["52.20.21.97:9100"]

  - job_name: 'jenkins'
    metrics_path: '/prometheus'
    static_configs:
      - targets: ['98.85.65.138:8080']
```

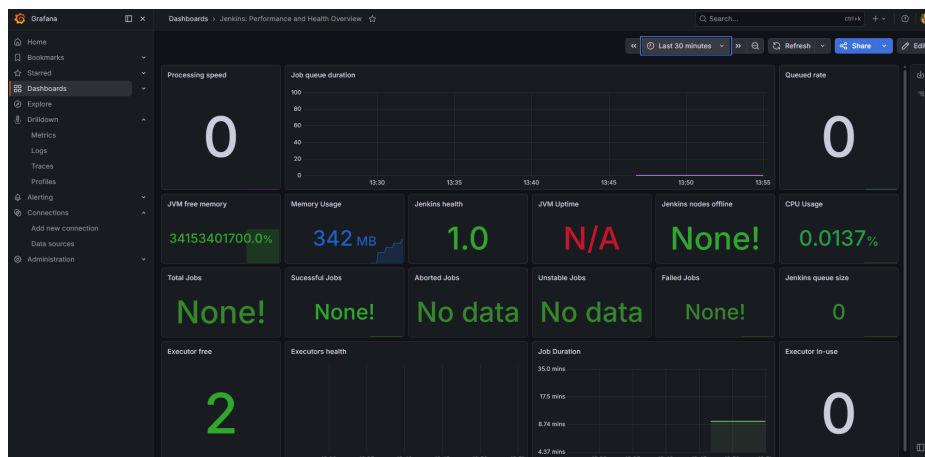
- Configure Jenkins metrics in prometheus



The screenshot shows the Prometheus web interface with the 'Targets' tab selected. It lists three scrape targets, all of which are 'UP'.

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
jenkins (1/1 up)					
http://98.85.65.138:8080/prometheus	UP	instance="98.85.65.138:8080" job="jenkins"	-38.274s ago	16.603ms	
node_exporter (1/1 up)					
http://52.20.21.97:9100/metrics	UP	instance="52.20.21.97:9100" job="node_exporter"	-34.561s ago	17.538ms	
prometheus (1/1 up)					
http://localhost:9090/metrics	UP	instance="localhost:9090" job="prometheus"	-29.496s ago	5.391ms	

- Import Dashboard ID 9964



Outcome

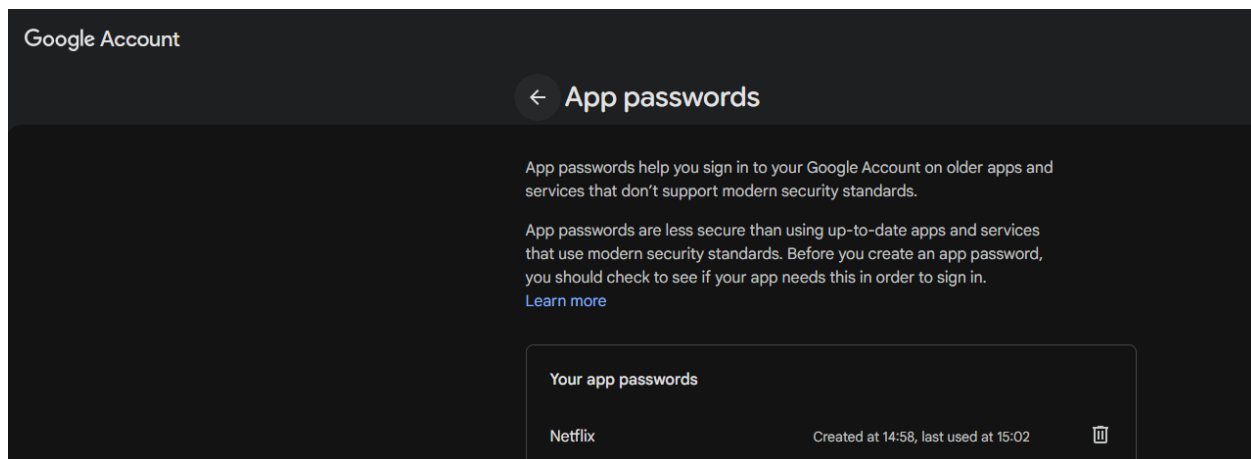
- ✓ Jenkins metrics visible in Grafana.

Phase 7: Notifications

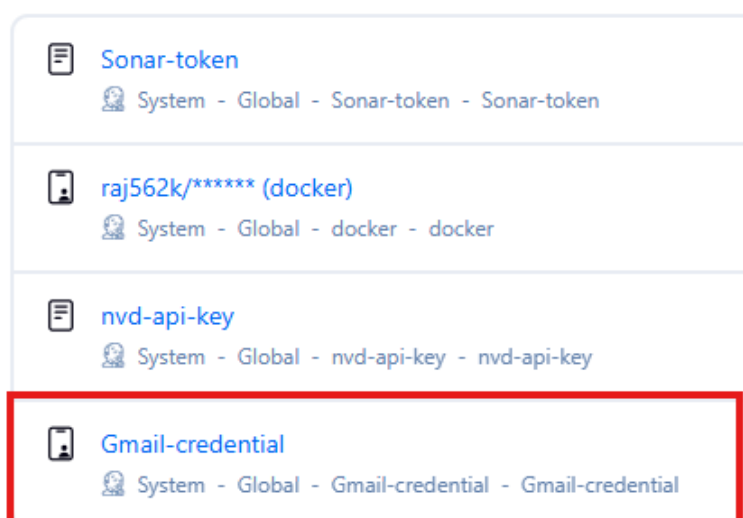
Step 21: Configure Email Notifications

Implementation

1. Enable Gmail 2FA.
2. Create App Password.
3. Add Gmail credentials to Jenkins.



Credentials





Default user e-mail suffix ?

[redacted]@gmail.com

! This field should be '@' followed by a domain name.

Advanced ^

Edited

☒ Use SMTP Authentication ?

User Name

[redacted]@gmail.com

! For security when using authentication it is recommended to enable either TLS or SSL

Password

.....

☒ Use SSL ?

☐ Use TLS

SMTP Port ?

465

Reply-To Address

Charset

UTF-8

Outcome

☒ Email delivery configured.

Step 22: Configure Extended Email Notifications

Extended E-mail Notification

SMTP server

smtp.gmail.com

SMTP Port

465

Advanced ^

 Edited

Credentials

raj562k/***** (docker)

☒ Use SSL

☐ Use TLS

☐ Use OAuth 2.0

Outcome

 Jenkins notification service operational.

Step 23: Add Email Block to Pipeline

```
post {
  always {
    emailx attachLog: true,
    subject: "${currentBuild.result}",
    body: "Project: ${env.JOB_NAME}<br/>" +
    "Build Number: ${env.BUILD_NUMBER}<br/>" +
    "URL: ${env.BUILD_URL}<br/>",
    to: 'mail@gmail.com',           #change mail here
    attachmentsPattern: 'trivyfs.txt,trivyimage.txt'
  }
}
```

Outcome

 Build notifications automated.

Step 24: Validate Email Delivery

Jenkins / netflix

Status netflix

Changes

Build Now

Configure

Delete Pipeline

Full Stage View

SonarQube

Stages

Rename

Pipeline Syntax

Dependency-Check Trend

Unassigned

Low

Medium

High

Critical

#5

#6

#7

#9

#10

#11

#14

Stage View

Average stage times:
(full run time: ~6min 46s)

	Declarative: Tool Install	clean workspace	Checkout from Git	Sonarqube Analysis	quality gate	Install Dependencies	OWASP FS SCAN	TRIVY FS SCAN	Patch Dockerfile	Docker Build & Push	TRIVY IMAGE SCAN	Deploy to container	Declarative: Post Actions
#16 Jun 13 1:01 PM	185ms	341ms	1s	27s	391ms	14s	4min 3s	2s	420ms	49s	588ms	663ms	758ms
#15 Jun 13 11:12 AM	177ms	324ms	1s	27s	378ms	15s	4min 41s	3s	548ms	1min 8s	789ms	1s	1s
#14 Jun 13 11:02 AM	188ms	361ms	1s	26s	373ms	18s	4min 44s	4s	559ms	1min 9s	806ms	1s	1s
#13 Jun 13 11:01 AM	172ms	302ms	799ms	25s	378ms	13s	4min 43s	4s	559ms	1min 8s	772ms	792ms	208ms
#12 Jun 13 10:53 AM	175ms	330ms	1s	26s	379ms	4s	78ms	74ms	77ms	78ms	77ms	74ms	176ms
#9 Jun 13 8:46 AM						aborted	aborted	aborted	aborted	aborted	aborted	aborted	
#8 Jun 11, 2026													

≡ Gmail

Compose

Inbox

Starred

Snoozed

Sent

Drafts

Purchases

More

Labels

More

Search mail

SUCCESS Inbox x

J

address not configured yet

to me

Project: netflix

Build Number: 16

URL: <http://98.85.65.138:8080/job/netflix/16/>

3 attachment • Scanned by Gmail

trivyfs.txt

trivyimage.txt

build.log

Reply

Forward

Outcome

✓ Email notifications received with trivy attachments.

25



Phase 8: Amazon EKS Deployment



Step 25: Create EKS Cluster

Step 1

Configure cluster

Step 2

Specify networking

Step 3

Configure observability

Step 4

Select add-ons

Step 5

Configure selected add-ons settings

Step 6

Review and create

Configure cluster

Configuration options

Choose how you would like to configure the cluster.

☐ Quick configuration (with EKS Auto Mode)

☒ Custom configuration

Quickly create a cluster with production-grade default settings. The configuration uses EKS Auto Mode to automate infrastructure tasks like creating nodes and provisioning storage.

To change default settings prior to creation, choose this option. This configuration gives the option to use EKS Auto Mode and customize the cluster's configuration.

EKS Auto Mode

Choose if you would like to use EKS's Auto Mode.

☒ Use EKS Auto Mode

EKS automates routine cluster tasks for compute, storage, and networking. When a new pod can't fit onto existing nodes, EKS creates a new node. EKS combines cluster infrastructure managed by AWS with integrated Kubernetes capabilities to meet application compute needs. [View pricing](#)

Included capabilities

Cluster configuration

Name

Enter a unique name for this cluster. This property cannot be changed after the cluster is created.

Netflix

The cluster name should begin with letter or digit, and can have any of the following characters: the set of Unicode letters, digits, hyphens and underscores. Maximum length of 100.

Cluster IAM role

To access other AWS services and resources your Kubernetes control plane needs an IAM role. Choose a role with the right permissions for your Kubernetes control plane.

eks-cluster-role

Create new role

Edit role

[View role details in IAM](#)

Kubernetes version settings

Kubernetes version

Select Kubernetes version for this cluster.

1.35

Configure cluster

Configuration options

Choose how you would like to configure the cluster.

☐ Quick configuration (with EKS Auto Mode)

☒ Custom configuration

Quickly create a cluster with production-grade default settings. The configuration uses EKS Auto Mode to automate infrastructure tasks like creating nodes and provisioning storage.

To change default settings prior to creation, choose this option. This configuration gives the option to use EKS Auto Mode and customize the cluster's configuration.

EKS Auto Mode

Choose if you would like to use EKS's Auto Mode.

☒ Use EKS Auto Mode

EKS automates routine cluster tasks for compute, storage, and networking. When a new pod can't fit onto existing nodes, EKS creates a new node. EKS combines cluster infrastructure managed by AWS with integrated Kubernetes capabilities to meet application compute needs. [View pricing](#)

Included capabilities

Cluster configuration

Name

Enter a unique name for this cluster. This property cannot be changed after the cluster is created.

Netflix

The cluster name should begin with letter or digit, and can have any of the following characters: the set of Unicode letters, digits, hyphens and underscores. Maximum length of 100.

Cluster IAM role

To access other AWS services and resources your Kubernetes control plane needs an IAM role. Choose a role with the right permissions for your Kubernetes control plane.

eks-cluster-role

Create new role

Edit role

[View role details in IAM](#)

Kubernetes version settings

Kubernetes version

Select Kubernetes version for this cluster.

1.35

Specify networking

Networking

IP address family and service IP address range cannot be changed after cluster creation.

VPC

Select a VPC to use for your EKS cluster resources.

vpc-0f642f69bbe91a20 | Default

Subnets

Choose the subnets in your VPC where the control plane may place elastic network interfaces (ENIs) to facilitate communication with your cluster. To create a new subnet, go to the corresponding page in the [VPC console](#).

Select subnets

subnet-0397528848a23ef4

subnet-09eb2903f5877a267

subnet-06809445cc40b8258

subnet-08b486079ffa194

subnet-0abba8f77175e9d0

Additional security groups

EKS automatically creates a cluster security group on cluster creation to facilitate communication between worker nodes and control plane. Optionally, choose additional security groups to apply to the EKS-managed Elastic Network interfaces that are created in your control plane subnets. To create a new security group, go to the corresponding page in the [VPC console](#).

Select security groups

sg-0b173eb5d46c6683 | default

Choose cluster IP address family

Specify the IP address type for pods and services in your cluster.

☒ IPv4

☐ IPv6

Configure Kubernetes service IP address block

☐ Specify the range from which cluster services will receive IP addresses.

Configure remote networks to enable hybrid nodes

EKS hybrid nodes enables you to use on-premises and edge infrastructure as nodes in EKS clusters.

☐ Specify the CIDR blocks for your on-premises environments that you will use for hybrid nodes.

Cluster endpoint access

Configure access to the Kubernetes API server endpoint.

☒ Public

☐ Public and private

☐ Private

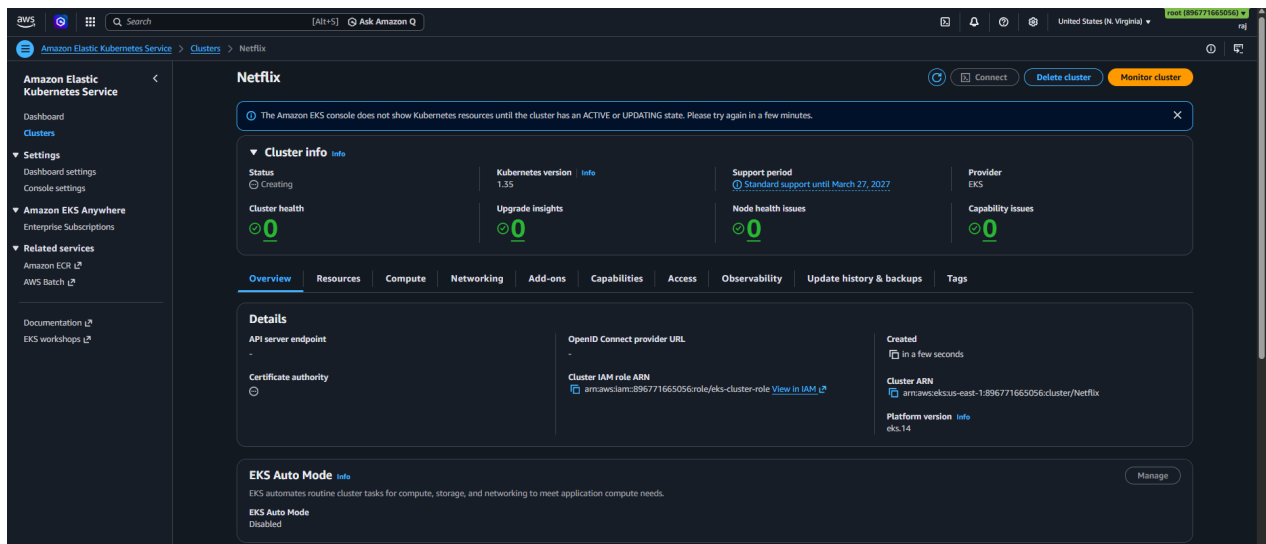
The cluster endpoint is accessible from outside of your VPC. Worker node traffic will leave your VPC to connect to the endpoint.

The cluster endpoint is accessible from outside of your VPC. Worker node traffic to the endpoint will stay within your VPC.

The cluster endpoint is only accessible through your VPC. Worker node traffic to the endpoint will stay within your VPC.

[Advanced settings](#)

26



Outcome

✅ EKS cluster provisioned.



Step 26: Configure Node Groups

Step 1

Configure node group

Step 2

Set compute and scaling configuration

Step 3

Specify networking

Step 4

Review and create

Configure node group

Info

A node group is a group of EC2 instances that supply compute capacity to your Amazon EKS cluster. You can add multiple node groups to your cluster.

Node group configuration

These properties cannot be changed after the node group is created.

Name

Assign a unique name for this node group.

nodes

The node group name should begin with letter or digit and can have any of the following characters: the set of Unicode letters, digits, hyphens and underscores. Maximum length of 63.

Node IAM role

To access other AWS services and resources your Nodes needs an IAM role. Choose a role with the right permissions for your Nodes.

eks-cluster-role

Create new role

Edit role

View role details in IAM

Step 1

Configure node group

Step 2

Set compute and scaling configuration

Step 3

Specify networking

Step 4

Review and create

Set compute and scaling configuration

Node group compute configuration

These properties cannot be changed after the node group is created.

AMI type Info
Select the EKS-optimized Amazon Machine Image for nodes.

Amazon Linux 2023 (x86_64) Standard (AL2023_x86_64_STANDARD)

Capacity type
Select the capacity purchase option for this node group.

On-Demand

Instance types Info
Select instance types you prefer for this node group.

Enter an instance type

t3.medium
vCPU: 2 vCPUs Memory: 4 GiB Network: Up to 5 Gigabit Max ENI: 3 Max IPs: 18

Disk size
Select the size of the attached EBS volume for each node.

20 GiB

Node group scaling configuration

Desired size
Set the desired number of nodes that the group should launch with initially.

1 nodes
Desired node size must be greater than or equal to 0

Minimum size
Set the minimum number of nodes that the group can scale in to.

1 nodes
Minimum node size must be greater than or equal to 0

Maximum size
Set the maximum number of nodes that the group can scale out to.

1 nodes
Maximum node size must be greater than or equal to 1 and cannot be lower than the minimum size

Amazon Elastic Kubernetes Service

Clusters

Netflix

nodes

Node group creation in progress

node is now being created. This process may take several minutes.

nodes

Node group configuration

Kubernetes version

1.35

AMI type

Amazon Linux 2023 (x86_64) Standard

Status

Creating

AMI release version

1.35.5-20230529

Instance types

t3.medium

Disk size

20 GiB

Details

Nodes

Health issues

Kubernetes labels

Update config

Kubernetes taints

Update history

Tags

Details

Node group ARN

arn:aws:eks:us-east-1:896771665056:nodegroup/Netflix/nodes/50c7f049-bc78-0f3a-4cca-aab4a2ad59ac

Autoscaling group name

aws-eks-asg-east-1-896771665056-nodegroup/Netflix/nodes/50c7f049-bc78-0f3a-4cca-aab4a2ad59ac

Capacity type

On-Demand

Subnets

subnet-039752884d8a25ef4

subnet-09ac2803f5877a2d7

subnet-0d5058a3ac4d5b7d18

subnet-06e8548d9798a194

subnet-0ab0d0ff7717ad9d0

Created

In a few seconds

Node IAM role ARN

arn:aws:iam::896771665056:role/AmazonEKS_CNI_Policy

Desired size

1 node

Minimum size

1 node

Maximum size

1 node

Configure remote access to nodes

Configure remote access to nodes

Node auto repair configuration

The node auto repair feature reacts to the Ready condition of the kubernetes and any node object manual deletions. It can detect more node conditions for repair when the node monitoring agent is also installed. Go to Add-ons

Node auto repair

Disabled

Warm pool

No warm pool currently configured.

Decrease scale-out latency by pre-initializing EC2 instances and save money by reducing the number of continuously running instances.

Outcome

✓ Worker nodes successfully attached.

Step 27: Configure Jenkins for EKS

Install

- AWS CLI
- kubectl
- eksctl

```
ubuntu@ip-172-31-29-176:~$ aws eks update-kubeconfig \
--region us-east-1 \
--name Netflix
Added new context arn:aws:eks:us-east-1:896771665056:cluster/Netflix to /home/ubuntu/.kube/co
nfig
ubuntu@ip-172-31-29-176:~$ kubectl get nodes
NAME                                STATUS    ROLES    AGE   VERSION
ip-172-31-72-183.ec2.internal      Ready    <none>   38m   v1.35.5-eks-3385e9b
ubuntu@ip-172-31-29-176:~$
```

Outcome

- ✓ Jenkins connected to Kubernetes cluster.

Phase 9: GitOps Deployment with Argo CD

Step 28: Install Argo CD

Implementation

1. Create Argo CD namespace.

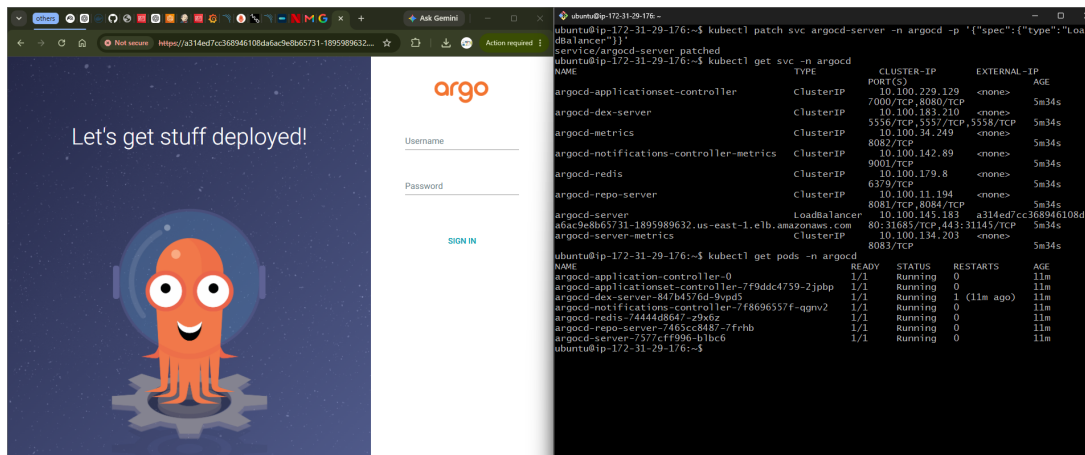
```
ubuntu@ip-172-31-29-176:~$ kubectl get pods -n argocd
NAME                                READY    STATUS    RESTARTS    AGE
argocd-application-controller-0     1/1      Running   0            46s
argocd-applicationset-controller-7f9ddc4759-2jpbp  1/1      Running   0            46s
argocd-dex-server-847b4576d-9vpd5    1/1      Running   1 (32s ago)  46s
argocd-notifications-controller-7f8696557f-qgnv2  1/1      Running   0            46s
argocd-redis-74444d8647-z9x6z        1/1      Running   0            46s
argocd-repo-server-7465cc8487-7frhb   1/1      Running   0            46s
argocd-server-7577c996-b1bc6         0/1      Running   0            46s
ubuntu@ip-172-31-29-176:~$ |
```

2. Expose Argo CD.

```
ubuntu@ip-172-31-29-176:~$ kubectl patch svc argocd-server -n argocd -p '{"spec":{"type":"LoadBalancer"}}'
service/argocd-server patched
ubuntu@ip-172-31-29-176:~$
```

```
ubuntu@ip-172-31-29-176:~$ kubectl get svc -n argocd
NAME                                TYPE                CLUSTER-IP      EXTERNAL-IP      AGE
argocd-applicationset-controller    ClusterIP           10.100.229.129  <none>           5m34s
argocd-dex-server                  ClusterIP           10.100.183.210  <none>           5m34s
argocd-metrics                    ClusterIP           10.100.34.249   <none>           5m34s
argocd-notifications-controller-metrics ClusterIP           10.100.142.89   <none>           5m34s
argocd-redis                      ClusterIP           10.100.179.8    <none>           5m34s
argocd-repo-server                 ClusterIP           10.100.11.194   <none>           5m34s
argocd-server                      LoadBalancer        10.100.145.183  a314ed7cc368946108d 5m34s
a6ac9e8b65731-1895989632.us-east-1.elb.amazonaws.com 80:31685/TCP,443:31145/TCP
argocd-server-metrics             ClusterIP           10.100.134.203  <none>           5m34s
8083/TCP
```

3. Argo CD server page.



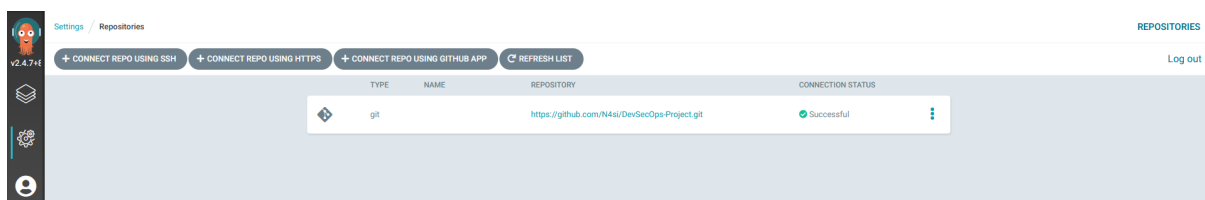
The screenshot shows the Argo CD web interface on a browser. The main area displays a login form with fields for 'Username' and 'Password', and a 'SIGN IN' button. To the left is a large illustration of the Argo CD octopus mascot. In the background, a terminal window is open, showing the execution of several kubectl commands to patch the argocd-server service and to list the pods and services in the argocd namespace. The terminal output confirms that the services are running and the pods are in a 'Running' state.

Outcome

✅ Argo CD available.

Step 29: Connect Git Repository

Created a app - which will fetch the data or manifest file present in the github repo k8s folder



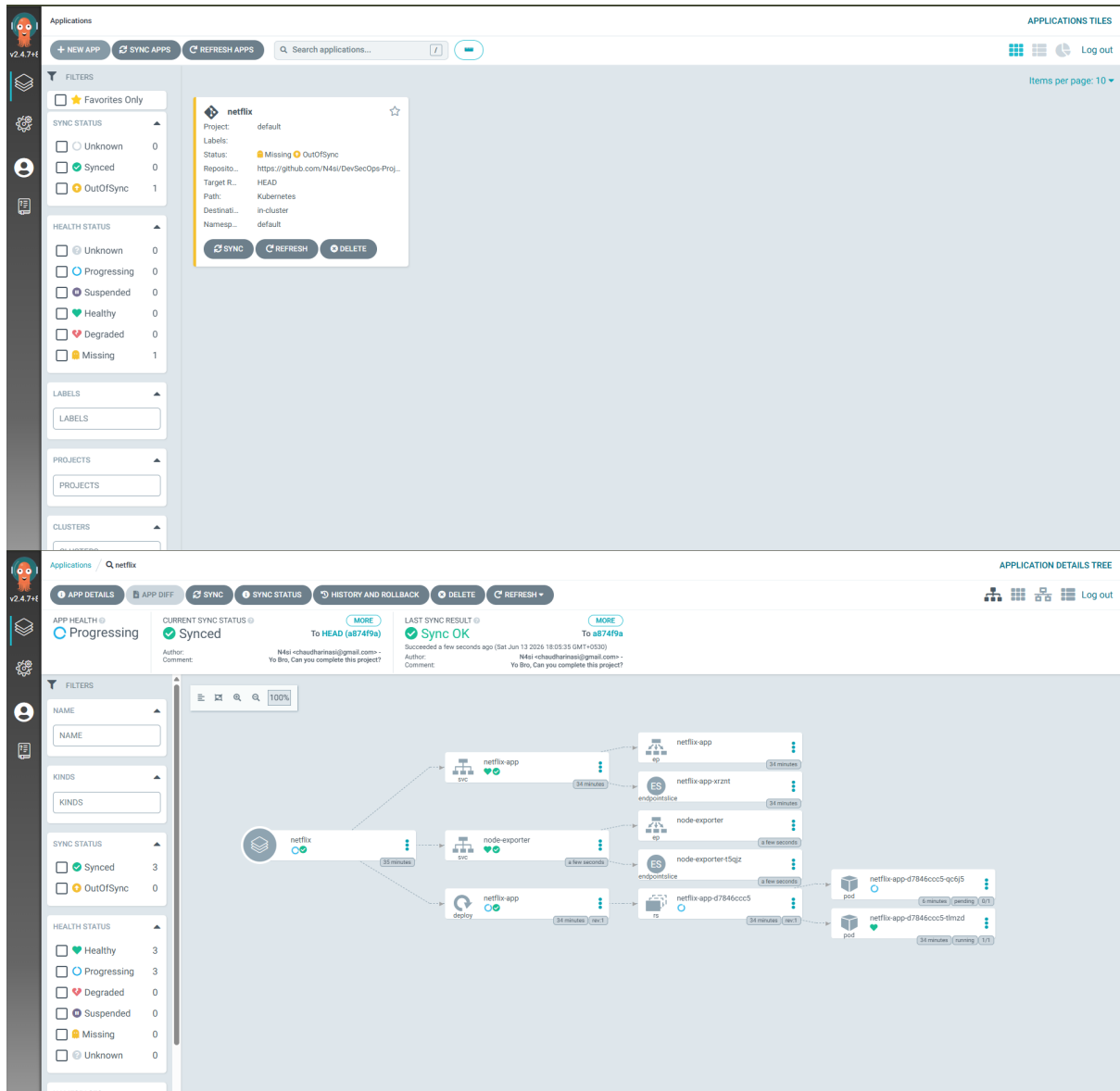
Outcome

✅ Git repository connected successfully.

Step 30: Create Argo CD Application

Purpose

Deploy Kubernetes manifests stored in the GitHub repository.



The screenshot displays the Argo CD web interface. The top navigation bar includes 'NEW APP', 'SYNC APPS', and 'REFRESH APPS' buttons. The left sidebar contains filters for 'SYNC STATUS' (Unknown, Synced, OutOfSync) and 'HEALTH STATUS' (Unknown, Progressing, Suspended, Healthy, Degraded, Missing). The main content area shows the 'netflix' application details. The 'CURRENT SYNC STATUS' is 'Synced' (green checkmark). The 'LAST SYNC RESULT' is 'Sync OK' (green checkmark). The deployment graph shows the following components and their sync status:

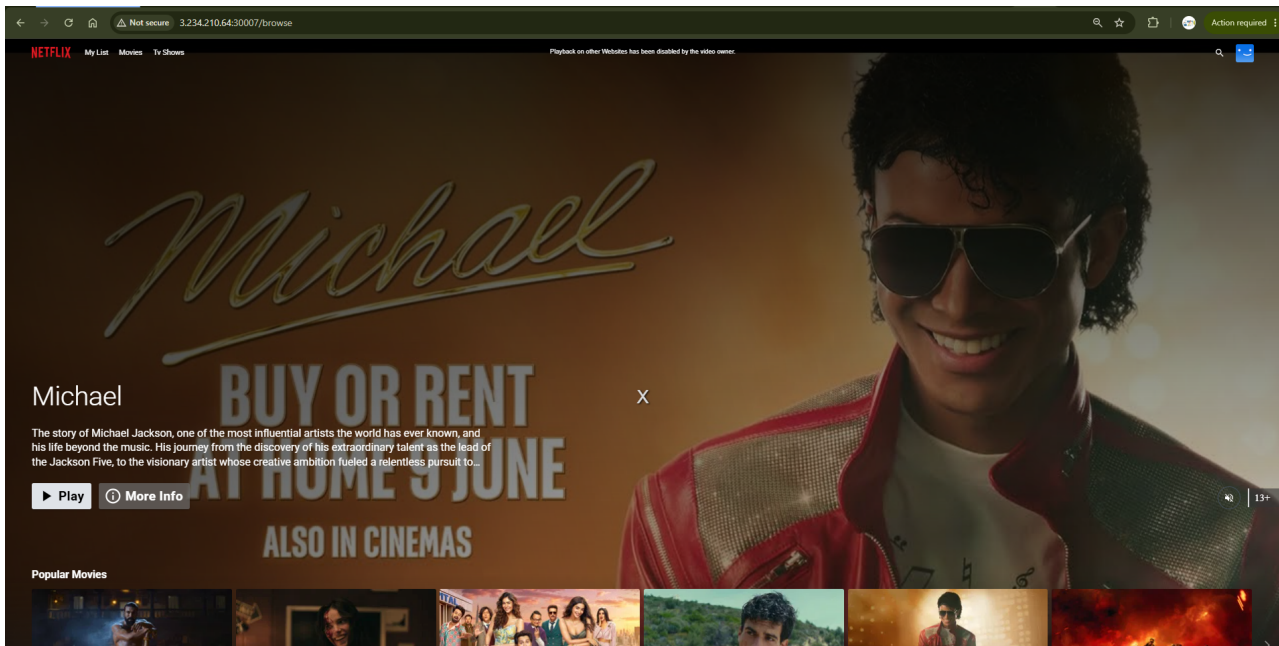
- netflix-app: Synced (34 minutes)
- netflix-app-xrztnt: Synced (34 minutes)
- node-exporter: Synced (a few seconds)
- node-exporter-t5qz: Synced (a few seconds)
- netflix-app-d7846ccc5-q6j5: Synced (34 minutes)
- netflix-app-d7846ccc5-timzd: Synced (34 minutes)

Outcome

 GitOps deployment configured.



Final Result:- Netflix Clone Successfully Deployed



Project Outcome Summary

Implementation Area	Status
 AWS Infrastructure Provisioning	 Completed
 Dockerized Application Deployment	 Completed
 SonarQube Static Code Analysis	 Completed
 Trivy Vulnerability Scanning	 Completed
 Jenkins CI/CD Pipeline Automation	 Completed
 OWASP Dependency Check Integration	 Completed
 Docker Hub Image Publishing	 Completed
 Prometheus, Grafana & Email Monitoring Setup	 Completed
 Amazon EKS Cluster Deployment	 Completed
 Argo CD GitOps Implementation	 Completed
 Netflix Clone Application Deployment	 Successfully Running

Final Result

The complete DevSecOps lifecycle was successfully implemented, covering infrastructure provisioning, security scanning, CI/CD automation, monitoring, alerting, Kubernetes orchestration, and GitOps-based deployment. The Netflix Clone application is fully deployed and operational on Amazon EKS using industry-standard DevSecOps practices.